An Analysis of Markov decision process

Markov decision process

Markov decision process -- Part 1

E [∑ t = 0 H − 1 R a t (s t , s t + 1)] {\displaystyle E\left[\sum _{t=0}^{H-1}{R_{a_{t}}(s_{t},s_{t+1})}\right]} (where we choose a t = π (s t) {\displaystyle a_{t}=\pi (s_{t})} , i.e. actions given by the policy). And the expectation is taken over s t + 1 ~ P a t (s t , s t + 1) {\displaystyle s_{t+1}\sim P_{a_{t}}(s_{t},s_{t+1})}

Markov decision process -- Part 2

∑ i ∈ S ∑ a ∈ A (i) R (i , a) y ∗ (i , a) ≥ ∑ i ∈ S ∑ a ∈ A (i) R (i , a) y (i , a) {\displaystyle {\begin{aligned}\sum _{i\in S}\sum _{a\in A(i)}R(i,a)y^{*}(i,a)\geq \sum _{i\in S}\sum _{a\in A(i)}R(i,a)y(i,a)\end{aligned}}}

Markov decision process -- Part 3

Policy iteration In policy iteration, one first performs Value Determination by solving for V {\displaystyle V} from the linear system described in step one, then performs Policy Improvement by computing π {\displaystyle \pi } as in step two, then repeats both steps until the policy converges. (Policy iteration was invented by Howard to optimize Sears catalogue mailing, which he had been optimizing using value iteration.) Since policy iteration effectively interleaves a linear inverse problem with a nonlinear operation, it may interpreted as a type of relaxation method. This variant has the advantage that there is a definite stopping condition. Since there is a unique solution V {\displaystyle V} for each policy π {\displaystyle \pi } , the algorithm is completed once the Policy Improvement produces the same policy twice consecutively. While there are situations where policy iteration may be faster than value iteration (e.g. when the action space is significantly larger than the state space), policy iteration is usually slower than value iteration for a large number of possible states.

Markov decision process -- Part 4

There are multiple costs incurred after applying an action instead of one. CMDPs are solved with linear programs only, and dynamic programming does not work. The final policy depends on the starting state. The method of Lagrange multipliers applies to CMDPs. Many Lagrangian-based algorithms have been developed.

Markov decision process -- Part 5

Maximize ∑ i ∈ S ∑ a ∈ A (i) R (i , a) y (i , a) s.t. ∑ i ∈ S ∑ a ∈ A (i) q (j ■ i , a) y (i , a) = 0 ∀ j ∈ S , ∑ i ∈ S ∑ a ∈ A (i) y (i , a) = 1 , y (i , a) ≥ 0 ∀ a ∈ A (i) and ∀ i ∈ S {\displaystyle {\begin{aligned}{\text{Maximize}}&\sum _{i\in S}\sum _{a\in A(i)}R(i,a)y(i,a)\\{\text{s.t.}}&\sum _{i\in S}\sum _{a\in A(i)}q(j\mid i,a)y(i,a)=0\quad \forall j\in S,\\&\sum _{i\in S}\sum _{a\in A(i)}y(i,a)=1,\\&y(i,a)\geq 0\qquad \forall a\in A(i){\text{ and }}\forall i\in S\end{aligned}}}

Markov decision process -- Part 6

V (s) := ∑ s ′ P π (s) (s , s ′) (R π (s) (s , s ′) + γ V (s ′)) {\displaystyle V(s):=\sum _{s'}P_{\pi (s)}(s,s')\left(R_{\pi (s)}(s,s')+\gamma V(s')\right)}

Markov decision process -- Part 7

Algorithms Solutions for MDPs with finite state and action spaces may be found through a variety of methods such as dynamic programming. The algorithms in this section apply to MDPs with finite state and action spaces and explicitly given transition probabilities and reward functions, but the basic concepts may be extended to handle other problem classes, for example using function approximation. Also, some processes with countably infinite state and action spaces can be exactly reduced to ones with finite state and action spaces. The standard family of algorithms to calculate optimal policies for finite state and action MDPs requires storage for two arrays indexed by state: value V {\displaystyle V} , which contains real values, and policy π {\displaystyle \pi } , which contains actions. At the end of the algorithm, π {\displaystyle \pi } will contain the solution and V (s) {\displaystyle V(s)} will contain the discounted sum of the rewards to be earned (on average) by following that solution from state s {\displaystyle s} . The algorithm has two steps, (1) a value update and (2) a policy update, which are repeated in some order for all the states until no further changes take place. Both recursively update a new estimation of the optimal policy and state value using an older estimation of those values.

Markov decision process -- Part 8

E [∑ t = 0 ∞ γ t R a t (s t , s t + 1)] {\displaystyle E\left[\sum _{t=0}^{\infty }{\gamma ^{t}R_{a_{t}}(s_{t},s_{t+1})}\right]} (where we choose a t = π (s t) {\displaystyle a_{t}=\pi (s_{t})} , i.e. actions given by the policy). And the expectation is taken over s t + 1 ~ P a t (s t , s t + 1) {\displaystyle s_{t+1}\sim P_{a_{t}}(s_{t},s_{t+1})}

Markov decision process -- Part 9

max E π ■ [∫ 0 ∞ γ t r (s (t) , π (s (t))) d t | s 0] {\displaystyle \max \operatorname {E} _{\pi }\left[\left.\int _{0}^{\infty }\gamma ^{t}r(s(t),\pi

(s(t)))\,dt\;\right|s_{0}\right]}

Markov decision process -- Part 10

While this function is also unknown, experience during learning is based on (s , a) {\displaystyle (s,a)} pairs (together with the outcome s ′ {\displaystyle s'} ; that is, "I was in state s {\displaystyle s} and I tried doing a {\displaystyle a} and s ′ {\displaystyle s'} happened"). Thus, one has an array Q {\displaystyle Q} and uses experience to update it directly. This is known as Q-learning.

Markov decision process -- Part 11

y (i , a) {\displaystyle y(i,a)} is a feasible solution to the D-LP if y (i , a) {\displaystyle y(i,a)} is nonnative and satisfied the constraints in the D-LP problem. A feasible solution y ∗ (i , a) {\displaystyle y^{*}(i,a)} to the D-LP is said to be an optimal solution if

Markov decision process -- Part 12

Simulator models In many cases, it is difficult to represent the transition probability distributions, P a (s , s ′) {\displaystyle P_{a}(s,s')} , explicitly. In such cases, a simulator can be used to model the MDP implicitly by providing samples from the transition distributions. One common form of implicit MDP model is an episodic environment simulator that can be started from an initial state and yields a subsequent state and reward every time it receives an action input. In this manner, trajectories of states, actions, and rewards, often called episodes may be produced. Another form of simulator is a generative model, a single step simulator that can generate samples of the next state and reward given any state and action. (Note that this is a different meaning from the term generative model in the context of statistical classification.) In algorithms that are expressed using pseudocode, G {\displaystyle G} is often used to represent a generative model. For example, the expression s ′ , r ← G (s , a) {\displaystyle s',r\gets G(s,a)} might denote the action of sampling from the generative model where s {\displaystyle s} and a {\displaystyle a} are the current state and action, and s ′ {\displaystyle s'} and r {\displaystyle r} are the new state and reward. Compared to an episodic simulator, a generative model has the advantage that it can yield data from any state, not only those encountered in a trajectory. These model classes form a hierarchy of information content: an explicit model trivially yields a generative model through sampling from the distributions, and repeated application of a generative model yields an episodic simulator. In the opposite direction, it is only possible to learn approximate models through regression. The type of model available for a particular MDP plays a significant role in determining which solution algorithms are appropriate. For example, the dynamic programming algorithms described in the next section require an explicit model, and Monte Carlo tree search requires a generative model (or an episodic simulator that can be copied at any state), whereas most reinforcement learning algorithms require only an episodic simulator.